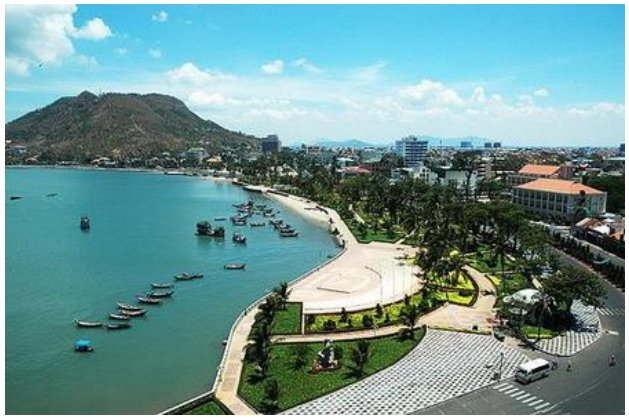
EMBEDDED SYSTEM

INTRODUCTION

HCMUTE – Faculty of Mechanical Engineering
Lecturer: PhD. Bui Ha Duc
Email: ducbh@hcmute.edu.vn

INSTRUCTOR INTRODUCTION





- Full Name: Bui Ha Duc
- From: Vung Tau City
- Bachelor: HCMC University of Technology
- Ph.D: National University of Singapore (NUS)
- Research fellow / TA in NUS
- Join HCMUTE in 2015
- Lecturer in Department of Mechatronics – FME
- Teaching: Microcontroller, Embedded system, Digital signal processing.

COURSE OBJECTIVES

- To understand the principles and concepts behind embedded systems

  MCU: **Raspberry**

- To obtain hands-on experience in design and programming embedded systems

- To practice Problem solving skill and team-work skill

ORDER OF INTRUCTION

- Introduction to embedded system
  - Architecture of embedded system.
  - Embedded system Hardware
  - Operating System for embedded system.
- Embedded system with Linux
  - Linux Kernel
  - Device driver
  - Compiler
  - Connecting peripheral devices

ORDER OF INTRUCTION

- Embedded system and internet of things
  - Wireless communication
  - Internet of things
  - Webserver
  - Control Hardware via webserver
- Image processing on Embedded board
  - Object detection
  - Object tracking

ASSESSMENTS

- Homework + Quizzes – **50%**
- Group Project – **50%**
  - Report – 10%
  - Presentation – 10%
  - Final Product – 30%

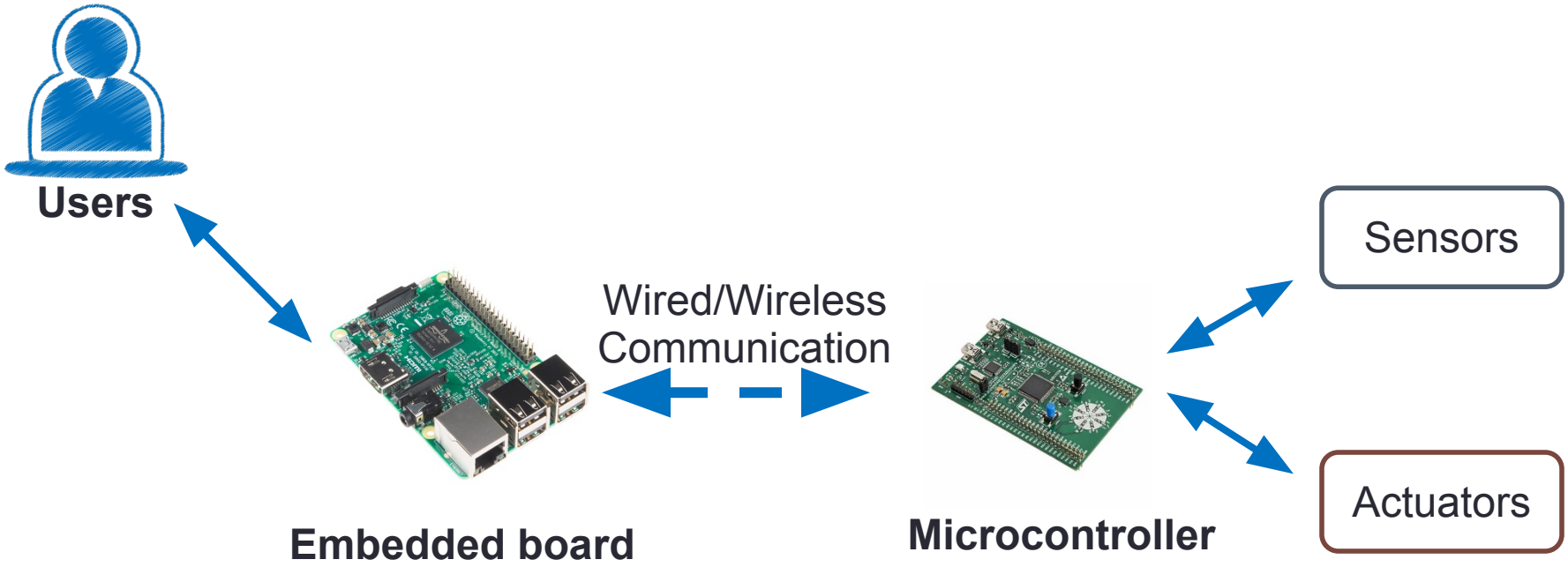


©Ron Leishman * illustrationsOf.com/440221

Team Selection

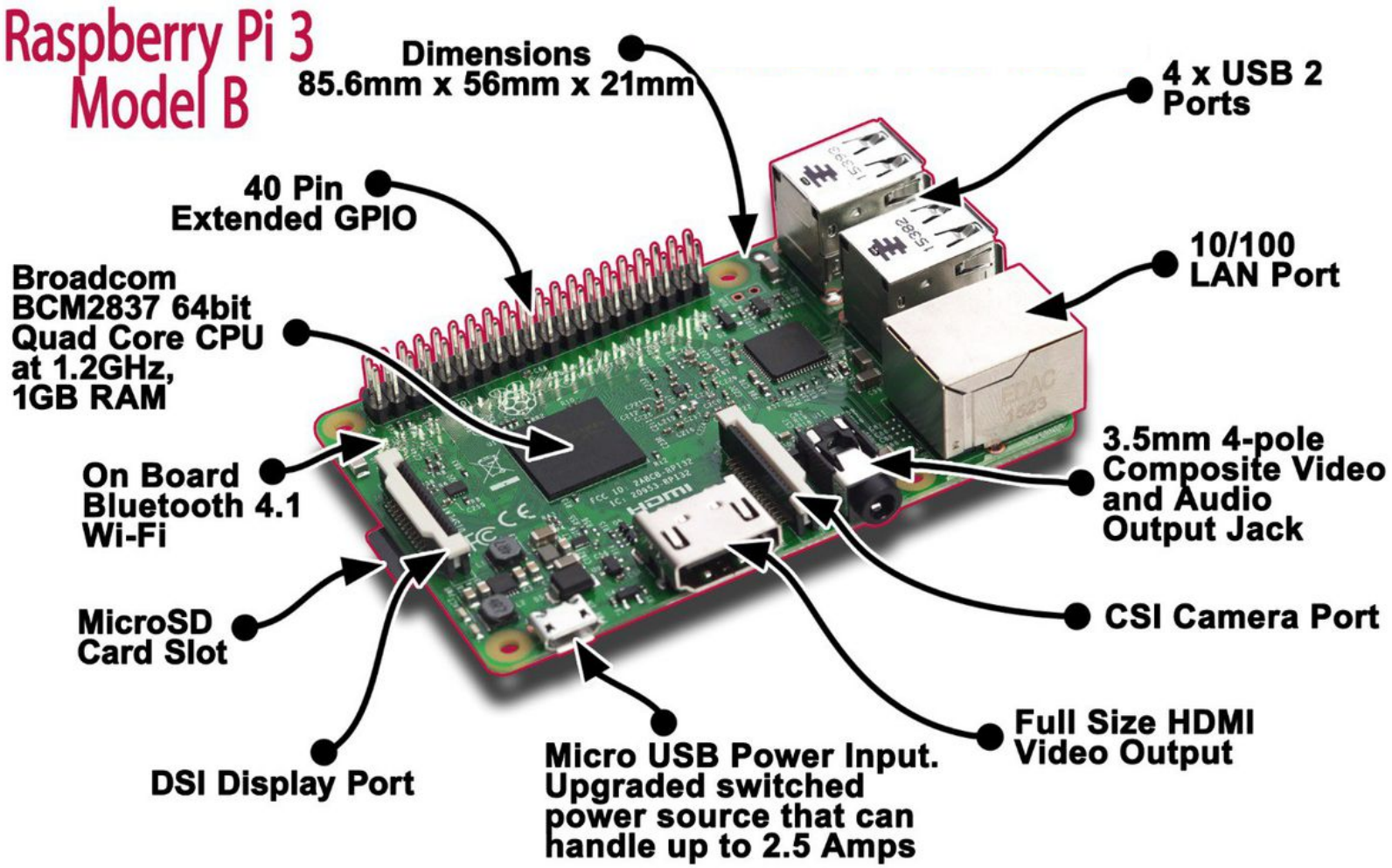
- Group of 3-4 students
- Group activities:
  - Discussion
  - Doing project
  - Presentation

Group project

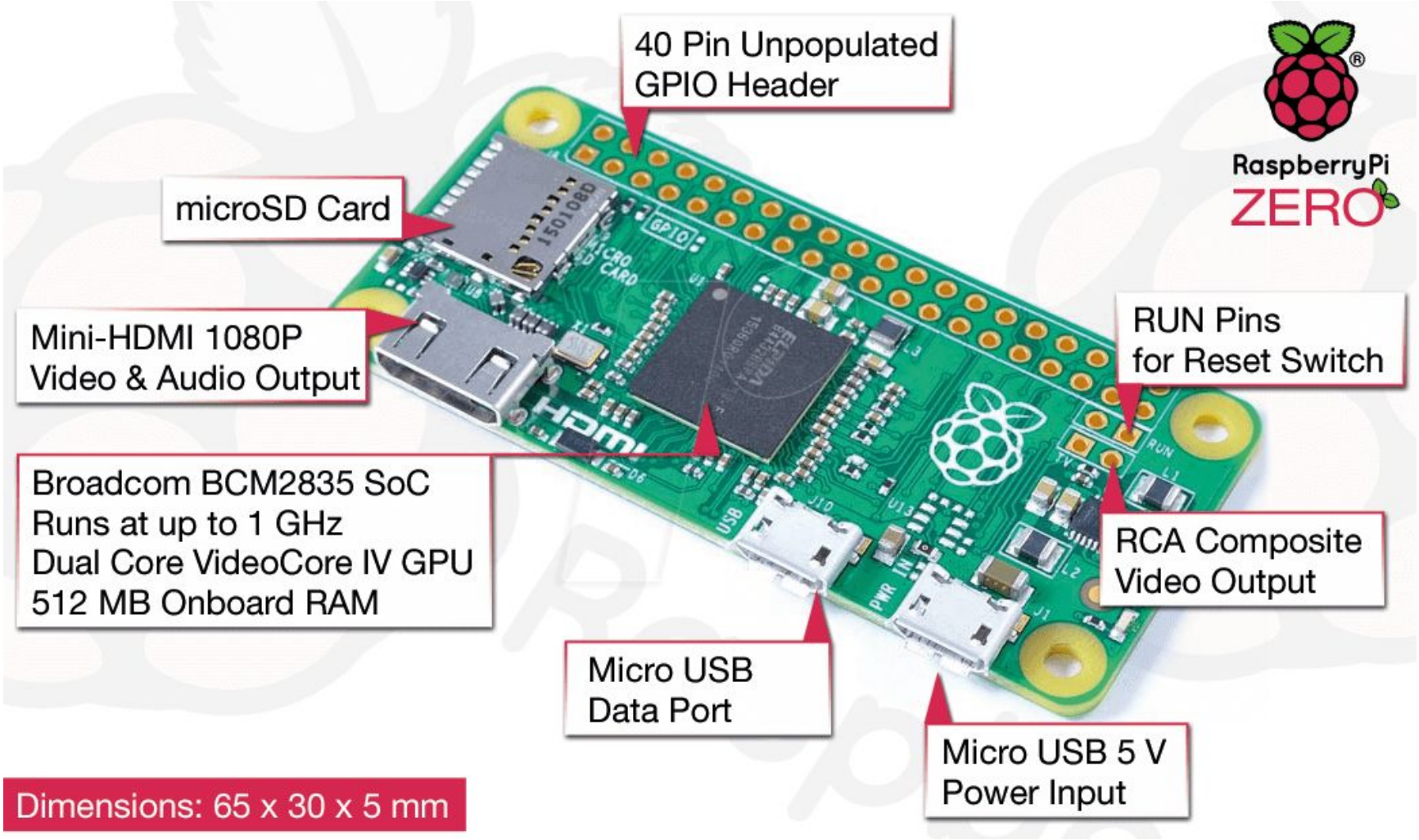


- Focus on IoT
- User control the hardware system via webserver
- Implement on Embedded board

Raspberry Pi Platform



Raspberry Pi Platform



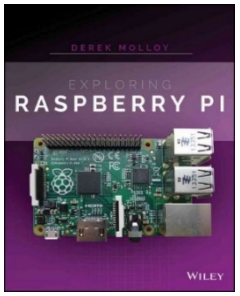
Raspberry emulator

- Setup tutorial

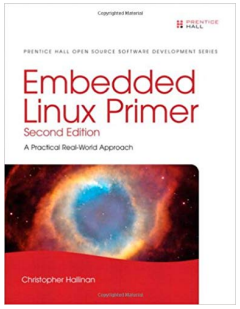
https://www.youtube.com/watch?v=xiQX0YXYuqU

- Package Required:
  - Emulator Qemu: http://qemu.weilnetz.de/w32/
  - RPi kernel : https://github.com/dhruvvyas90/qemu-rpi-kernel
  - Raspbain image: http://downloads.raspberrypi.org/

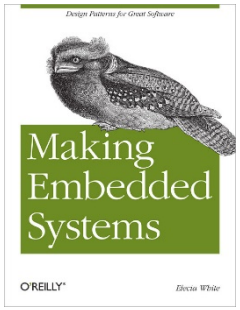
References



- Derek Molloy, **Exploring Raspberry Pi**, John Wiley & Sons, 2016.
  - web: *http://exploringrpi.com/*
  - Source code: *https://github.com/derekmolloy/exploringrpi.git*

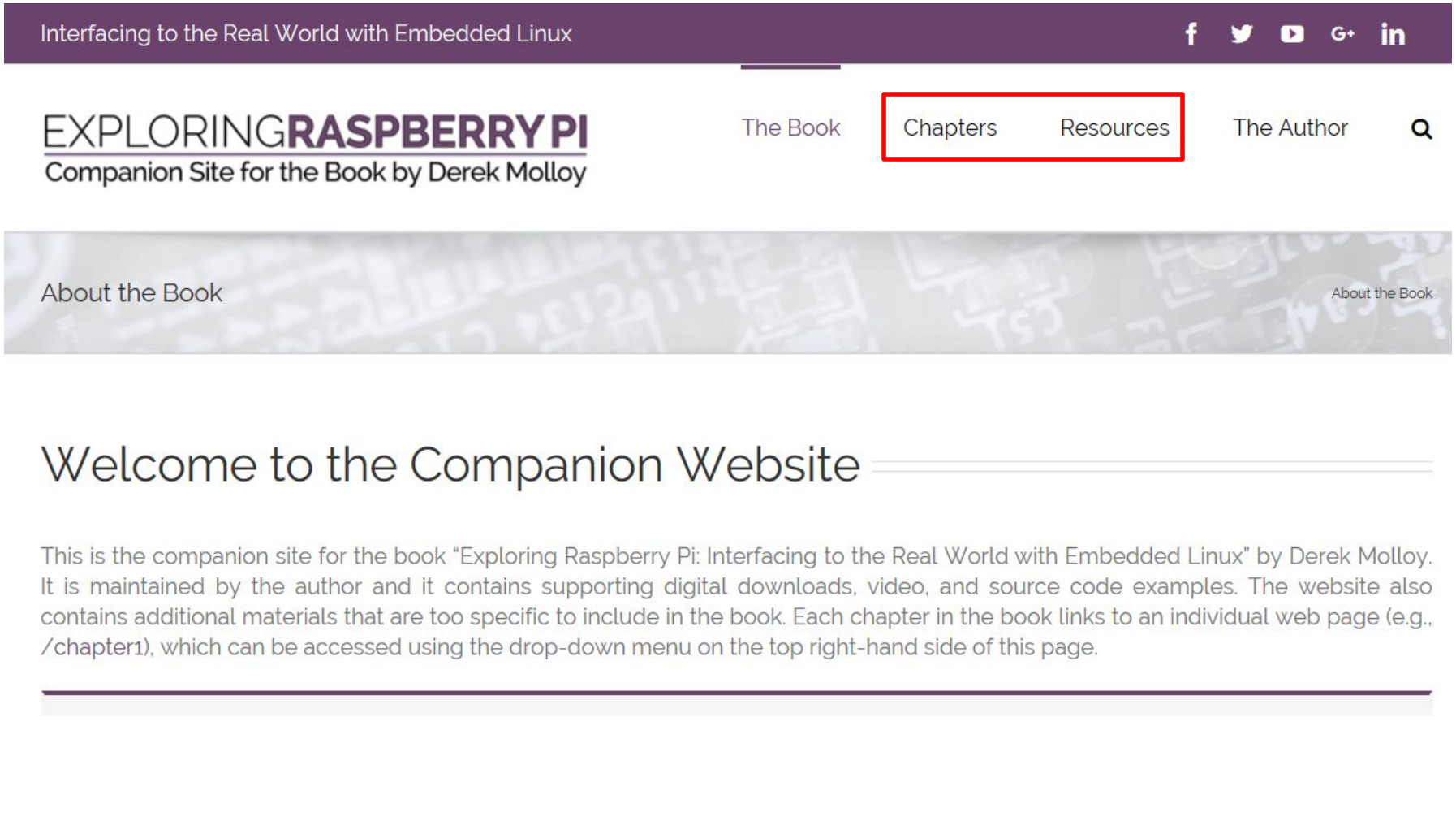


- Christopher Hallinan, **Embedded Linux Primer**, 2nd edition, Prentice Hall 2011.



- Elecia White, **Making Embedded Systems**, O'Reilly, 2011.

Exploring Raspberry Pi

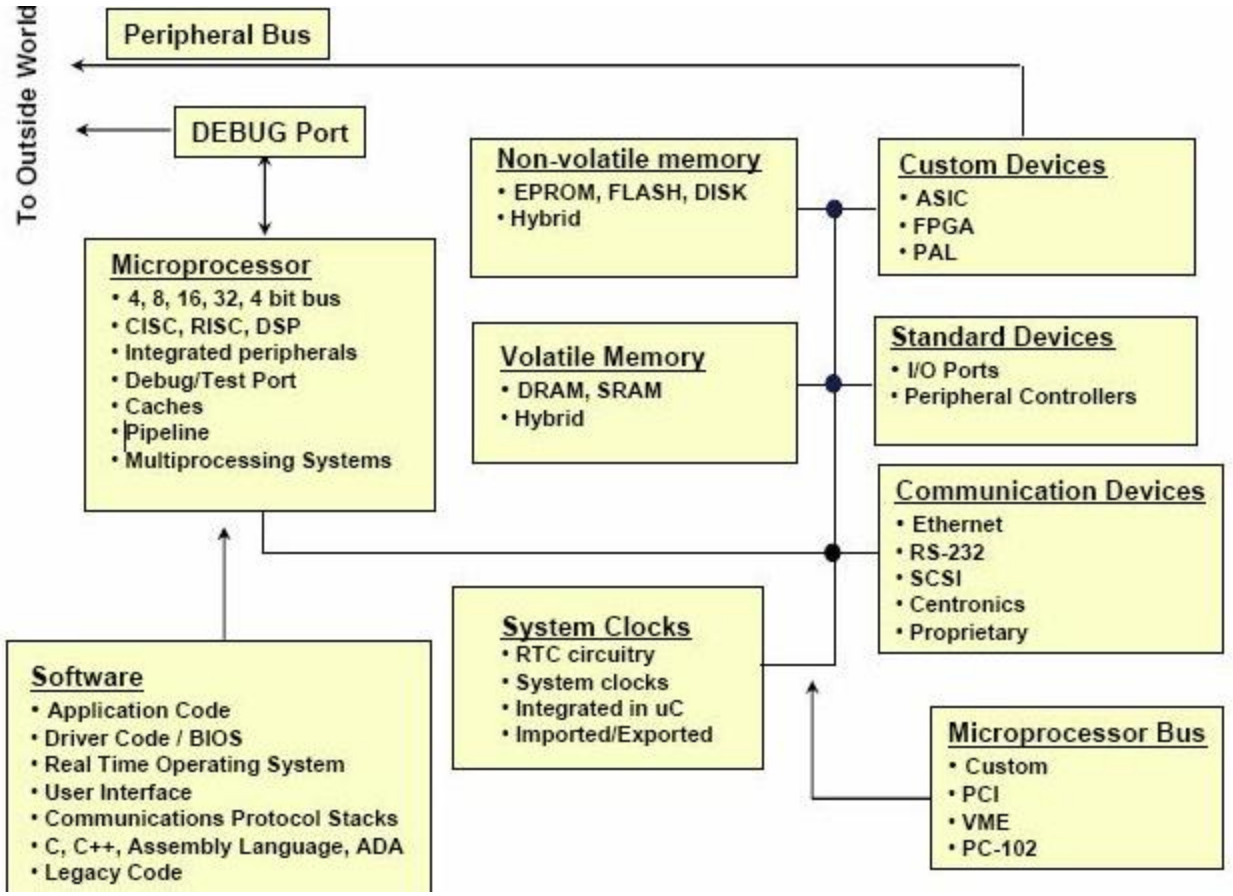


What is an embedded system?

- An embedded system is a **computerized system** that is **purpose built** for its application.
  - Small device/component of a larger machine
  - Have small processor installed
  - Have software that control the device
  - Must have real-time response

- Embedded systems are where the software meets the physical world.

Embedded system architecture



Embedded system vs Computer



Computer system

- General purpose
- Powerful hardware
- Convenient to develop software
- Event-based action



Embedded system

- Specialized purpose
- Hardware has constrains
- Difficult to write software
- Response in real-time
- Interfacing with hardware

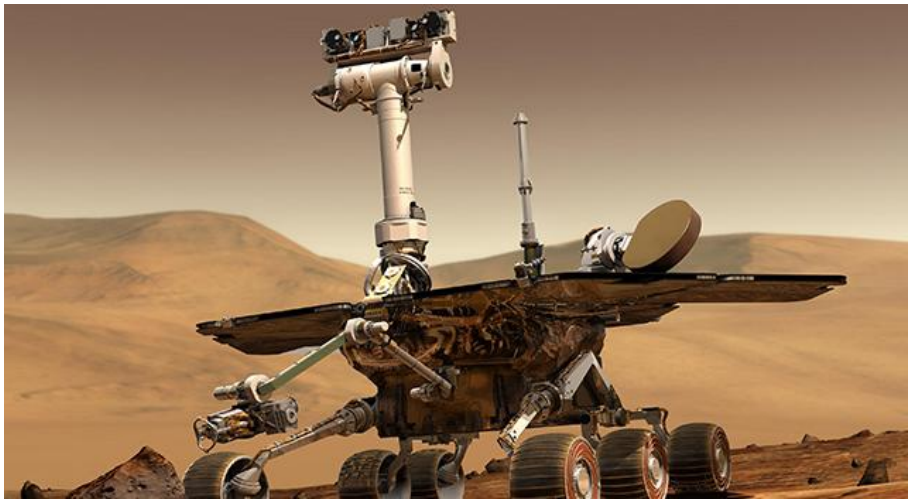
Examples of Embedded system

- TV
- stereo
- remote control
- phone / mobile phone
- refrigerator
- microwave
- electric tooth brush
- oven / rice or bread cooker
- watch
- alarm clock
- electronic musical instruments
- electronic toys (stuffed animals,handheld toys, pinballs, etc.)
- medical home equipment (e.g. bloodpressure, thermometer)

Examples of Embedded system

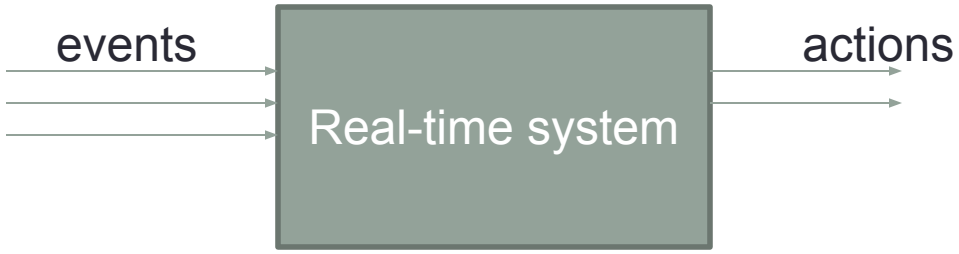






What is a real-time system (RTS)?

- A RTS is an event driven system
- Events can be generated externally or internally
- Deadline must be satisfied in a timely manner



Real-time ≠ Fast

- A real-time system is not a fast system
- Speed is relative to specific environment
- Correct behavior is better than running faster

Reliability in Embedded system

- It's difficult to debug an embedded system
- Even final product can be suffered from bugs
  - Transistor size becoming smaller – 14nm!
    - Harder to fabricate
    - Yield in semiconductor is decreasing
  - As a result … more susceptible to fault →
    - e.g PS3 CPU has 8 processors → only 6 to 7 processors working
- Design a reliable, fault tolerant embedded system is crucial → using modularity

Embedded Operating system

- Not all embedded systems have operating systems
  - E.g. Microwave oven, washing machine

- An operating system creates the ability to:
  - serve a variety of purposes
  - interact with users in more complicated ways
  - keep up with needs that change over time

Operating system

Basically, an operating system does two things:

- It manages the hardware and software resources of the system.

- It provides a stable, consistent way for applications to deal with the hardware without having to know all the details of the hardware.
  - API – Application program interface

Types of Operating Systems

- **Real-time operating system** (RTOS)
- **Single-user, single task**
  - Palm, android
- **Single-user, multi-tasking**
  - Windows, MacOS
- **Multi-user**

Embedded OS

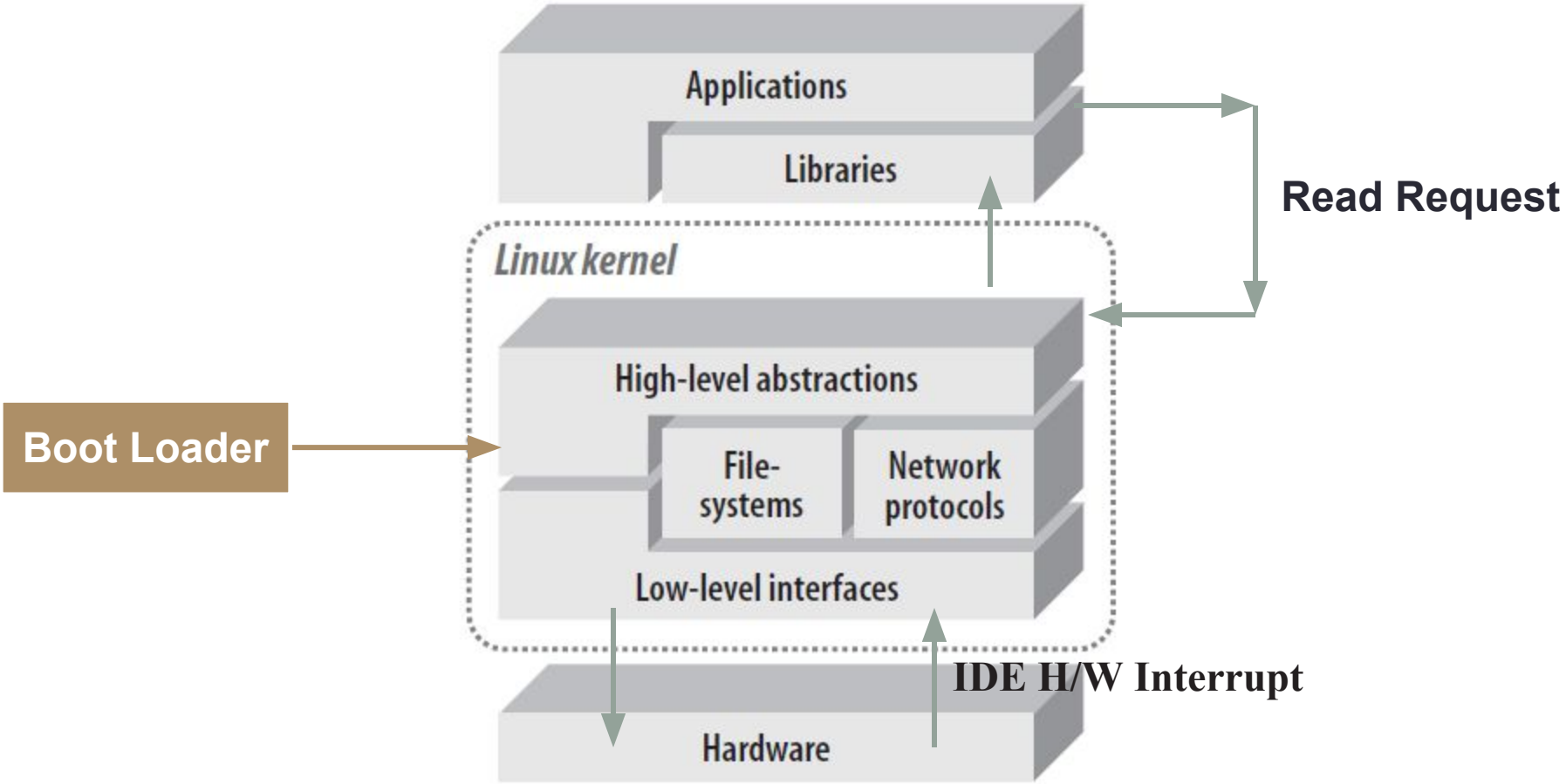
- Embedded operating system is typically quite limited in terms of function
  - E.g. traffic lights, digital televisions, ATMs, digital cameras, GPS navigation systems, elevators
- An embedded OS must be reliable and able to run with constraints on memory, size and processing power.

Embedded Linux

Why Linux?

- Linux supports a vast variety of hardware devices, probably more than any other OS.
- Linux supports a huge variety of applications and networking protocols.
- Linux has attracted a huge number of active developers, enabling rapid support of new hardware architectures, platforms, and devices.
- An increasing number of hardware and software vendors support Linux.

Generic Architecture of an Embedded Linux System



Homework

- Install VirtualBox or Vmware on your computer
- Install Ubuntu (32bit) on the virtual machine
- Download raspberry OS

  https://www.raspberrypi.org/downloads/raspbian/
- Each group Buy a MicroSD card
- Read ebook chapter 1
  - **Raspberry Pi Hardware**